

OS Lab 2

Q1) Copy the following codes in a C file and execute them to observe the behaviour of fork, exec, wait and exit system calls.

FORK1.C:

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

int main(int argc, char * argv[]) {

printf("hello world (pid:%d)\n", (int) getpid());

int rc = fork();

if (rc < 0) { // fork failed; exit

fprintf(stderr, "fork failed\n");
1
exit(1);

} else if (rc == 0) { // child (new process)

printf("hello, I am child (pid:%d)\n", (int) getpid());

} else { // parent goes down this path (main)

printf("hello, I am parent of %d (pid:%d)\n",

rc, (int) getpid());

}

return 0;

}
```

FORK 2:

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>
```

```

#include <sys/wait.h>

int main(int argc, char * argv[]) {

printf("hello world (pid:%d)\n", (int) getpid());

int rc = fork();

if (rc < 0) { // fork failed; exit
fprintf(stderr, "fork failed\n");
exit(1);
} else if (rc == 0) { // child (new process)
printf("hello, I am child (pid:%d)\n", (int) getpid());
} else { // parent goes down this path (main)
int rc_wait = wait(NULL);
printf("hello, I am parent of %d (rc_wait:%d) (pid:%d)\n",
rc, rc_wait, (int) getpid());
}

return 0;
}

```

FORK3:

```

#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <string.h>

#include <sys/wait.h>

int main(int argc, char * argv[]) {

printf("hello world (pid:%d)\n", (int) getpid());

```

```

int rc = fork();

if (rc < 0) { // fork failed; exit
    fprintf(stderr, "fork failed\n");
    exit(1);
} else if (rc == 0) { // child (new process)
    printf("hello, I am child (pid:%d)\n", (int) getpid());
    char *myargs[3];
    myargs[0] = strdup("wc"); // program: "wc" (word count)
    myargs[1] = strdup("p3.c"); // argument: file to count
    myargs[2] = NULL; // marks end of array
    execvp(myargs[0], myargs); // runs word count
    printf("this shouldn't print out");
} else { // parent goes down this path (main)
    int rc_wait = wait(NULL);
    printf("hello, I am parent of %d (rc_wait:%d) (pid:%d)\n",
        rc, rc_wait, (int) getpid());
}

return 0;
}

```

FORK4 with redirection:

```

#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <string.h>

#include <fcntl.h>

#include <sys/wait.h>

```

```

int main(int argc, char *argv[]) {

int rc = fork();

if (rc < 0) { // fork failed; exit

fprintf(stderr, "fork failed\n");

exit(1);

} else if (rc == 0) { // child: redirect standard output to a file

close(STDOUT_FILENO);

open("./p4.output", O_CREAT|O_WRONLY|O_TRUNC, S_IRWXU
);

// now exec "wc"...

char *myargs[3];

myargs[0] = strdup("wc"); // program: "wc" (word count)

myargs[1] = strdup("p4.c"); // argument: file to count

myargs[2] = NULL; // marks end of array

execvp(myargs[0], myargs); // runs word count

} else { // parent goes down this path (main)

int rc_wait = wait(NULL);

}

return 0;
}

```

Q. 2. Write a program that calls fork(). Before calling fork(), have the main process access a variable (e.g., x) and set its value to something (e.g., 100). What value is the variable in the child process? What happens to the variable when both the child and parent change the value of x ?

Q.3. Write a slight modification of the previous program, this time using waitpid() instead of wait(). When would waitpid() be useful?